



UNIVERSITÉ ALIOUNE DIOP DE BAMBEY
DÉPARTEMENT TECHNOLOGIES DE L'INFORMATION ET DE LA
COMMUNICATION
MASTER 1 - SYSTÈMES D'INFORMATION

Cours Complet Spring Boot *De Débutant à Expert*



Spring

Rédigé par
Modou DIAGNE
Étudiant en Master 1 Systèmes d'Information

Email: diagnemodou7199@gmail.com

Tél: +221 78 464 37 40

Année Universitaire 2025-2026

Table des matières

Introduction	11
I Fondamentaux de Spring Boot	13
1 Prérequis Java	15
1.1 Programmation Orientée Objet (POO)	15
1.1.1 Les 4 piliers de la POO	15
1.1.2 Exemple : Encapsulation	15
1.1.3 Exemple : Interfaces et Polymorphisme	16
1.2 Java Collections Framework	17
1.2.1 Les principales collections	17
1.3 Lambda et Stream API (Java 8+)	18
1.3.1 Expressions Lambda	18
1.3.2 Stream API	18
1.4 Optional (Java 8+)	19
1.5 Annotations Java	20
1.6 Questions d'entretien	20
2 Introduction à Spring et Spring Boot	23
2.1 Qu'est-ce que Spring ?	23
2.1.1 Problèmes résolus par Spring	23
2.1.2 Inversion de Contrôle (IoC)	23
2.2 Qu'est-ce que Spring Boot ?	24
2.2.1 Spring vs Spring Boot	24
2.2.2 Avantages de Spring Boot	24
2.3 Architecture d'une application Spring Boot	25
2.4 Création d'un projet Spring Boot	25
2.4.1 Avec Spring Initializr	25
2.4.2 Structure du projet	25
2.4.3 Le fichier pom.xml	26
2.4.4 Classe principale	27
2.5 Premier endpoint REST	28
2.5.1 Tester l'application	28
2.6 Questions d'entretien	29

II	Développement Web REST	31
3	API REST - Concepts et Principes	33
3.1	Qu'est-ce qu'une API REST ?	33
3.1.1	Les 6 contraintes REST	33
3.1.2	Méthodes HTTP	33
3.1.3	Codes de statut HTTP	34
3.1.4	Design des URLs RESTful	34
3.2	Format JSON	34
4	Controllers et Services Spring	37
4.1	Annotations des Controllers	37
4.1.1	@RestController	37
4.1.2	Mapping des requêtes	37
4.1.3	ResponseEntity	38
4.2	La couche Service	39
4.2.1	Responsabilités du Service	39
4.3	DTOs (Data Transfer Objects)	41
4.4	Validation avec Bean Validation	42
4.5	Questions d'entretien	43
III	Persistance des Données avec JPA	45
5	JPA et Hibernate - Fondamentaux	47
5.1	Introduction à JPA	47
5.1.1	Qu'est-ce que JPA ?	47
5.1.2	Avantages de JPA	47
5.2	Configuration Spring Data JPA	47
5.2.1	Dépendances Maven	47
5.2.2	Configuration application.properties	48
5.3	Les Entités JPA	49
5.3.1	Création d'une entité	49
5.3.2	Annotations JPA principales	51
5.3.3	Stratégies de génération d'ID	51
5.4	Spring Data JPA Repositories	51
5.4.1	JpaRepository	51
5.4.2	Query Methods - Mots-clés	53
5.4.3	Requêtes JPQL personnalisées	53
5.5	Pagination et Tri	54
6	Relations JPA	57
6.1	Types de relations	57
6.2	Relation OneToMany / ManyToOne	57
6.3	Relation ManyToMany	58
6.4	Fetch Types	59
6.5	Cascade Types	60
6.6	Questions d'entretien	60

IV	Sécurité avec Spring Security	63
7	Spring Security - Fondamentaux	65
7.1	Introduction à Spring Security	65
7.1.1	Dépendance Maven	65
7.2	Architecture de Spring Security	65
7.2.1	Composants principaux	66
7.3	Configuration de base	66
7.4	UserDetailsService	67
7.4.1	Entité User implémentant UserDetails	68
7.5	Autorisation par méthode	69
8	Authentification JWT	71
8.1	Introduction à JWT	71
8.1.1	Structure d'un JWT	71
8.2	Implémentation JWT	72
8.2.1	Dépendances	72
8.2.2	Configuration	72
8.2.3	Service JWT	72
8.2.4	Filtre JWT	74
8.2.5	Configuration Security avec JWT	76
8.3	Controller d'authentification	77
8.4	Questions d'entretien	78
V	Production et Bonnes Pratiques	81
9	Documentation API avec OpenAPI	83
9.1	Introduction à OpenAPI/Swagger	83
9.1.1	Dépendance	83
9.1.2	Configuration	83
9.1.3	Configuration Java	83
9.2	Annotations OpenAPI	84
9.2.1	Documentation des DTOs	85
10	Bonnes Pratiques de Production	87
10.1	Structure de projet	87
10.2	Gestion des erreurs	88
10.3	Profils Spring	89
10.4	Logging	90
11	Spring Boot Actuator	93
11.1	Configuration	93
11.2	Endpoints Actuator	93
11.3	Health Indicator personnalisé	94
11.4	Questions d'entretien	94

A	Cheatsheet Spring Boot	97
A.1	Annotations principales	97
A.1.1	Configuration et Composants	97
A.1.2	Mapping HTTP	97
A.1.3	JPA / Hibernate	98
A.1.4	Validation	98
A.1.5	Spring Security	98
A.2	Commandes Maven	98
A.3	Configuration application.properties	99
A.4	Template de projet	100
A.4.1	Entité de base	100
A.4.2	Repository	101
A.4.3	Service	101
A.4.4	Controller	102
B	Glossaire	103
C	Ressources supplémentaires	105
C.1	Documentation officielle	105
C.2	Tutoriels recommandés	105
C.3	Outils	105
	Références	107

Table des figures

2.1	Inversion de Contrôle	24
2.2	Architecture en couches d'une application Spring Boot	25
5.1	Fonctionnement de JPA	47
7.1	Architecture Spring Security	65
8.1	Flux d'authentification JWT	72

Liste des tableaux

1.1	Les 4 piliers de la POO	15
1.2	Collections Java les plus utilisées	17
2.1	Comparaison Spring Framework vs Spring Boot	24
3.1	Les 6 contraintes de l'architecture REST	33
3.2	Méthodes HTTP et leurs usages	33
3.3	Codes HTTP courants	34
3.4	Bonnes pratiques pour les URLs REST	34
5.1	Options de hibernate.ddl-auto	48
5.2	Annotations JPA essentielles	51
5.3	Mots-clés pour les Query Methods	53
6.1	Types de relations JPA	57
6.2	Stratégies de chargement	59
6.3	Types de cascade	60
7.1	Composants de Spring Security	66
10.1	Niveaux de log	91
11.1	Endpoints Actuator principaux	93

Introduction

Ce cours a été conçu pour fournir une formation complète et progressive sur **Spring Boot**, le framework Java le plus populaire pour le développement d'applications d'entreprise et de microservices.

À qui s'adresse ce cours ?

Ce cours est destiné à :

- Les étudiants en informatique souhaitant maîtriser le développement backend
- Les développeurs Java débutants ou intermédiaires
- Les professionnels préparant des entretiens techniques
- Toute personne souhaitant créer des API REST professionnelles

Prérequis

- Connaissances de base en programmation Java (OOP, Collections)
- Notions de base en SQL
- Familiarité avec un IDE (IntelliJ IDEA ou VS Code recommandé)

Structure du cours

Le cours est organisé en 5 parties progressives :

1. **Fondamentaux** : Introduction à Spring et Spring Boot
2. **Développement Web** : Création d'API REST
3. **Persistance des données** : JPA et Hibernate
4. **Sécurité** : Spring Security et JWT
5. **Production** : Bonnes pratiques et monitoring

Conseil

Chaque chapitre contient des exercices pratiques et des questions d'entretien pour consolider vos connaissances.

Première partie

Fondamentaux de Spring Boot

Chapitre 1

Prérequis Java

Avant de commencer avec Spring Boot, il est essentiel de maîtriser certains concepts Java modernes qui sont largement utilisés dans le framework.

1.1 Programmation Orientée Objet (POO)

1.1.1 Les 4 piliers de la POO

TABLE 1.1 – Les 4 piliers de la POO

Pilier	Description	Exemple
Encapsulation	Cacher les détails internes	Attributs private + getters/setters
Héritage	Réutiliser le code d'une classe parente	<code>class Dog extends Animal</code>
Polymorphisme	Un objet peut prendre plusieurs formes	Interface avec plusieurs implémentations
Abstraction	Définir un contrat sans implémentation	Classes abstraites et interfaces

1.1.2 Exemple : Encapsulation

```
1 public class User {
2     // Attributs privés (encapsulation)
3     private Long id;
4     private String email;
5     private String password;
6
7     // Constructeur
8     public User(String email, String password) {
9         this.email = email;
10        this.password = password;
11    }
12
13    // Getters
```

```
14     public Long getId() { return id; }
15     public String getEmail() { return email; }
16
17     // Setters avec validation
18     public void setEmail(String email) {
19         if (email != null && email.contains("@")) {
20             this.email = email;
21         }
22     }
23 }
```

Listing 1.1 – Exemple d’encapsulation en Java

1.1.3 Exemple : Interfaces et Polymorphisme

```
1 // Interface - contrat
2 public interface PaymentService {
3     void processPayment(double amount);
4     boolean refund(String transactionId);
5 }
6
7 // Implementation 1
8 public class StripePaymentService implements PaymentService {
9     @Override
10    public void processPayment(double amount) {
11        System.out.println("Paielement Stripe: " + amount);
12    }
13
14    @Override
15    public boolean refund(String transactionId) {
16        // Logique de remboursement Stripe
17        return true;
18    }
19 }
20
21 // Implementation 2
22 public class PayPalPaymentService implements PaymentService {
23     @Override
24    public void processPayment(double amount) {
25        System.out.println("Paielement PayPal: " + amount);
26    }
27
28    @Override
29    public boolean refund(String transactionId) {
30        // Logique de remboursement PayPal
31        return true;
32    }
33 }
```

Listing 1.2 – Interfaces en Java

Information

Spring Boot utilise massivement les interfaces pour permettre l'injection de dépendances et le découplage des composants.

1.2 Java Collections Framework

1.2.1 Les principales collections

TABLE 1.2 – Collections Java les plus utilisées

Interface	Implémentation	Caractéristiques
List	ArrayList	Liste ordonnée, accès par index, doublons permis
List	LinkedList	Liste chaînée, insertion/suppression rapide
Set	HashSet	Ensemble sans doublons, non ordonné
Set	TreeSet	Ensemble trié, sans doublons
Map	HashMap	Clé-valeur, accès $O(1)$, non ordonné
Map	TreeMap	Clé-valeur, clés triées

```

1 import java.util.*;
2
3 public class CollectionsDemo {
4     public static void main(String[] args) {
5         // List - elements ordonnes avec doublons
6         List<String> fruits = new ArrayList<>();
7         fruits.add("Pomme");
8         fruits.add("Banane");
9         fruits.add("Pomme"); // Doubleton autorise
10
11        // Set - elements uniques
12        Set<String> uniqueFruits = new HashSet<>(fruits);
13        // Resultat: [Pomme, Banane]
14
15        // Map - paires cle-valeur
16        Map<String, Integer> stock = new HashMap<>();
17        stock.put("Pomme", 100);
18        stock.put("Banane", 50);
19
20        // Iteration
21        for (Map.Entry<String, Integer> entry : stock.entrySet()) {
22            System.out.println(entry.getKey() + ": " + entry.
23                               getValue());
24        }
25    }

```

Listing 1.3 – Utilisation des collections

1.3 Lambda et Stream API (Java 8+)

1.3.1 Expressions Lambda

Les lambdas permettent d'écrire des fonctions anonymes de manière concise.

```
1 // Sans lambda (classe anonyme)
2 Comparator<String> comparatorOld = new Comparator<String>() {
3     @Override
4     public int compare(String s1, String s2) {
5         return s1.length() - s2.length();
6     }
7 };
8
9 // Avec lambda
10 Comparator<String> comparatorNew = (s1, s2) -> s1.length() - s2.
    length();
11
12 // Interfaces fonctionnelles courantes
13 Consumer<String> printer = s -> System.out.println(s);
14 Predicate<Integer> isPositive = n -> n > 0;
15 Function<String, Integer> length = s -> s.length();
16 Supplier<Double> random = () -> Math.random();
```

Listing 1.4 – Expressions Lambda

1.3.2 Stream API

```
1 List<User> users = Arrays.asList(
2     new User("alice@mail.com", 25),
3     new User("bob@mail.com", 30),
4     new User("charlie@mail.com", 22)
5 );
6
7 // Filtrer et transformer
8 List<String> adultEmails = users.stream()
9     .filter(u -> u.getAge() >= 18)           // Filtrer
10    .map(User::getEmail)                     // Transformer
11    .sorted()                               // Trier
12    .collect(Collectors.toList());           // Collecter
13
14 // Trouver un element
15 Optional<User> found = users.stream()
16    .filter(u -> u.getEmail().equals("bob@mail.com"))
17    .findFirst();
18
19 // Statistiques
20 double averageAge = users.stream()
21    .mapToInt(User::getAge)
22    .average()
23    .orElse(0.0);
```

```
24
25 // Grouper par critere
26 Map<Boolean, List<User>> byAge = users.stream()
27     .collect(Collectors.groupingBy(u -> u.getAge() >= 25));
```

Listing 1.5 – Stream API - Opérations courantes

Conseil

Les Streams sont **paresseux** (lazy) : les opérations intermédiaires ne sont exécutées que lorsqu'une opération terminale est appelée.

1.4 Optional (Java 8+)

Optional permet de gérer proprement les valeurs potentiellement nulles.

```
1 public class UserService {
2
3     public Optional<User> findByEmail(String email) {
4         User user = userRepository.findByEmail(email);
5         return Optional.ofNullable(user);
6     }
7
8     public void processUser(String email) {
9         // Mauvaise pratique (avant Optional)
10        User user = findByEmail(email);
11        if (user != null) {
12            System.out.println(user.getName());
13        }
14
15        // Bonne pratique (avec Optional)
16        findByEmail(email)
17            .map(User::getName)
18            .ifPresent(System.out::println);
19
20        // Valeur par défaut
21        String name = findByEmail(email)
22            .map(User::getName)
23            .orElse("Utilisateur inconnu");
24
25        // Lever une exception si absent
26        User user = findByEmail(email)
27            .orElseThrow(() -> new UserNotFoundException(email));
28    }
29 }
```

Listing 1.6 – Utilisation d'Optional

Attention

Ne jamais appeler `.get()` sur un `Optional` sans vérifier sa présence avec `.isPresent()` ou utiliser les méthodes `.orElse()`, `.orElseThrow()`.

1.5 Annotations Java

Les annotations sont omniprésentes dans Spring Boot.

```

1 // Annotation standard
2 @Override // Indique qu'on redefinit une methode
3 public String toString() {
4     return "User{email=" + email + "}";
5 }
6
7 @Deprecated // Marque comme obsolete
8 public void oldMethod() { }
9
10 @SuppressWarnings("unchecked") // Supprime les avertissements
11 public void method() { }
12
13 // Annotations Spring (aperçu)
14 @RestController // Controleur REST
15 @Service // Service metier
16 @Repository // Acces aux donnees
17 @Component // Composant generique
18 @Autowired // Injection de dependance
19 @Configuration // Classe de configuration

```

Listing 1.7 – Annotations Java courantes

1.6 Questions d'entretien

Question d'entretien

Q1 : Quelle est la différence entre une classe abstraite et une interface ?

Réponse :

- **Classe abstraite** : peut avoir des attributs, des méthodes concrètes et abstraites. Une classe ne peut hériter que d'une seule classe abstraite.
- **Interface** : depuis Java 8, peut avoir des méthodes default et static. Une classe peut implémenter plusieurs interfaces.

En Spring, on préfère les interfaces pour le découplage et l'injection de dépendances.

Question d'entretien

Q2 : Quelle est la différence entre `==` et `.equals()` ?

Réponse :

- `==` compare les références (adresses mémoire)
- `.equals()` compare le contenu (doit être redéfini)

```
1 String s1 = new String("hello");
2 String s2 = new String("hello");
3 s1 == s2;           // false (references différentes)
4 s1.equals(s2);      // true (contenu identique)
```

Question d'entretien

Q3 : Qu'est-ce qu'un Stream en Java ?

Réponse : Un Stream est une séquence d'éléments supportant des opérations de traitement fonctionnel (filter, map, reduce). Les streams sont :

- **Paresseux** : calcul différé jusqu'à l'opération terminale
- **Non réutilisables** : un stream ne peut être consommé qu'une fois
- **Potentiellement infinis** : `Stream.iterate()`, `Stream.generate()`

Chapitre 2

Introduction à Spring et Spring Boot

2.1 Qu'est-ce que Spring ?

Spring Framework est un framework Java open-source créé en 2003 par Rod Johnson. Il fournit une infrastructure complète pour le développement d'applications Java d'entreprise.

2.1.1 Problèmes résolus par Spring

1. **Couplage fort** : Sans Spring, les classes créent directement leurs dépendances, rendant le code difficile à tester et maintenir.
2. **Code boilerplate** : Beaucoup de code répétitif pour les tâches communes (transactions, sécurité, etc.).
3. **Configuration complexe** : Les applications Java EE nécessitaient des fichiers XML volumineux.

2.1.2 Inversion de Contrôle (IoC)

L'IoC est le principe fondamental de Spring : au lieu de créer vos dépendances, vous les recevez.

```
1 // SANS Spring - Couplage fort
2 public class OrderService {
3     private PaymentService paymentService;
4
5     public OrderService() {
6         // Creation directe = couplage fort
7         this.paymentService = new StripePaymentService();
8     }
9 }
10
11 // AVEC Spring - Injection de dependances
12 @Service
13 public class OrderService {
14     private final PaymentService paymentService;
15
16     // Spring injecte automatiquement l'implementation
```

```

17  @Autowired
18  public OrderService(PaymentService paymentService) {
19      this.paymentService = paymentService;
20  }
21  }

```

Listing 2.1 – Sans Spring vs Avec Spring

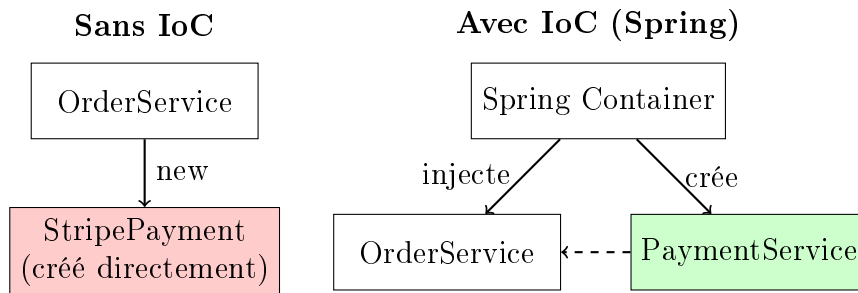


FIGURE 2.1 – Inversion de Contrôle

2.2 Qu'est-ce que Spring Boot ?

Spring Boot est une extension de Spring Framework qui simplifie la création d'applications Spring en éliminant la configuration complexe.

2.2.1 Spring vs Spring Boot

TABLE 2.1 – Comparaison Spring Framework vs Spring Boot

Aspect	Spring Framework	Spring Boot
Configuration	Manuelle (XML ou Java)	Auto-configuration
Serveur	Déploiement sur Tomcat externe	Serveur embarqué
Dépendances	Gestion manuelle des versions	Starters pré-configurés
Démarrage	Configuration longue	Démarrage en minutes
Production	Configuration manuelle	Actuator intégré

2.2.2 Avantages de Spring Boot

1. **Auto-configuration** : Spring Boot configure automatiquement l'application en fonction des dépendances présentes.
2. **Starters** : Des ensembles de dépendances pré-configurées.
3. **Serveur embarqué** : Tomcat, Jetty ou Undertow intégré.
4. **Pas de XML** : Configuration 100% Java ou YAML.
5. **Production-ready** : Métriques, health checks avec Actuator.

2.3 Architecture d'une application Spring Boot

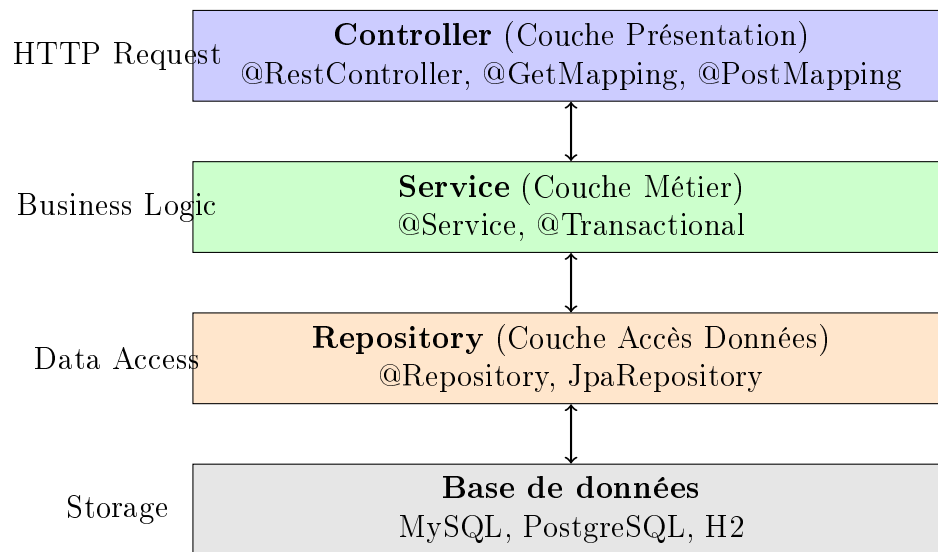


FIGURE 2.2 – Architecture en couches d'une application Spring Boot

2.4 Création d'un projet Spring Boot

2.4.1 Avec Spring Initializr

1. Aller sur <https://start.spring.io>
2. Configurer :
 - Project : Maven
 - Language : Java
 - Spring Boot : 3.2.x
 - Group : com.example
 - Artifact : demo
 - Java : 17
3. Ajouter les dépendances :
 - Spring Web
 - Spring Data JPA
 - H2 Database
4. Cliquer sur "Generate"

2.4.2 Structure du projet

Listing 2.2 – Structure d'un projet Spring Boot

```
mon-projet/  
|-- src/  
|   |-- main/
```

```

|   |   |-- java/
|   |   |   |-- com/example/demo/
|   |   |       |-- DemoApplication.java
|   |   |       |-- controller/
|   |   |       |-- service/
|   |   |       |-- repository/
|   |   |       |-- entity/
|   |   |-- resources/
|   |       |-- application.properties
|   |       |-- static/
|   |       |-- templates/
|   |-- test/
|       |-- java/
|-- pom.xml

```

2.4.3 Le fichier pom.xml

Listing 2.3 – pom.xml minimal

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
  </parent>

  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0.0</version>
  <name>Demo Application</name>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <!-- Spring Web -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Data JPA -->
    <dependency>

```

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- H2 Database -->
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- Test -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>
```

2.4.4 Classe principale

```
1 package com.example.demo;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class DemoApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(DemoApplication.class, args);
11     }
12 }
```

Listing 2.4 – Classe principale Spring Boot

Information

@SpringBootApplication est une annotation composite qui combine :

- @Configuration : Classe de configuration Spring
- @EnableAutoConfiguration : Active l'auto-configuration
- @ComponentScan : Scanne les composants dans le package courant

2.5 Premier endpoint REST

```
1 package com.example.demo.controller;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class HelloController {
8
9     @GetMapping("/hello")
10    public String hello() {
11        return "Hello, Spring Boot!";
12    }
13
14    @GetMapping("/hello/{name}")
15    public String helloName(@PathVariable String name) {
16        return "Hello, " + name + "!";
17    }
18 }
```

Listing 2.5 – Premier Controller REST

2.5.1 Tester l'application

Listing 2.6 – Lancer et tester l'application

```
# Lancer l'application
mvn spring-boot:run

# Tester avec curl
curl http://localhost:8080/hello
# Reponse: Hello, Spring Boot!

curl http://localhost:8080/hello/Modou
# Reponse: Hello, Modou!
```

2.6 Questions d'entretien

Question d'entretien

Q1 : Qu'est-ce que l'Inversion de Contrôle (IoC) ?

Réponse : L'IoC est un principe où le contrôle de la création et gestion des objets est transféré au framework (conteneur Spring) au lieu du code applicatif. Avantages :

- Découplage des composants
- Facilité de test (injection de mocks)
- Configuration centralisée

Question d'entretien

Q2 : Quelle est la différence entre @Component, @Service, @Repository ?

Réponse : Ce sont tous des stéréotypes Spring qui marquent une classe comme bean géré par Spring :

- **@Component** : Composant générique
- **@Service** : Logique métier (couche service)
- **@Repository** : Accès aux données (gestion des exceptions de persistance)
- **@Controller** : Contrôleur MVC

Fonctionnellement équivalents, mais la distinction sémantique améliore la lisibilité.

Question d'entretien

Q3 : Qu'est-ce que @SpringBootApplication ?

Réponse : C'est une méta-annotation qui combine :

- **@Configuration** : Indique une classe de configuration
- **@EnableAutoConfiguration** : Configure automatiquement les beans selon les dépendances
- **@ComponentScan** : Scanne le package courant et ses sous-packages

Deuxième partie

Développement Web REST

Chapitre 3

API REST - Concepts et Principes

3.1 Qu'est-ce qu'une API REST ?

REST (REpresentational State Transfer) est un style d'architecture pour concevoir des services web. Une **API REST** permet à des applications de communiquer via le protocole HTTP.

3.1.1 Les 6 contraintes REST

TABLE 3.1 – Les 6 contraintes de l'architecture REST

Contrainte	Description
Client-Serveur	Séparation des responsabilités entre client et serveur
Sans état (Stateless)	Chaque requête contient toutes les informations nécessaires
Cacheable	Les réponses peuvent être mises en cache
Interface uniforme	URLs cohérentes, méthodes HTTP standard
Système en couches	Le client ne sait pas s'il communique directement avec le serveur
Code à la demande	(Optionnel) Le serveur peut envoyer du code exécutable

3.1.2 Méthodes HTTP

TABLE 3.2 – Méthodes HTTP et leurs usages

Méthode	Action	Description	Idempotent
GET	Read	Récupérer une ressource	Oui
POST	Create	Créer une nouvelle ressource	Non
PUT	Update	Remplacer complètement une ressource	Oui
PATCH	Partial Update	Modifier partiellement une ressource	Non
DELETE	Delete	Supprimer une ressource	Oui

Idempotent signifie que plusieurs appels identiques produisent le même résultat. GET, PUT et DELETE sont idempotents ; POST ne l'est pas (crée une nouvelle ressource à chaque appel).

3.1.3 Codes de statut HTTP

TABLE 3.3 – Codes HTTP courants

Code	Nom	Usage
2xx - Succès		
200	OK	Requête réussie (GET, PUT)
201	Created	Ressource créée (POST)
204	No Content	Succès sans contenu (DELETE)
4xx - Erreur client		
400	Bad Request	Requête invalide (validation)
401	Unauthorized	Non authentifié
403	Forbidden	Non autorisé
404	Not Found	Ressource non trouvée
409	Conflict	Conflit (doublon)
5xx - Erreur serveur		
500	Internal Server Error	Erreur serveur
503	Service Unavailable	Service indisponible

3.1.4 Design des URLs RESTful

TABLE 3.4 – Bonnes pratiques pour les URLs REST

Méthode	URL	Description
GET	/api/users	Liste tous les utilisateurs
GET	/api/users/{id}	Récupère un utilisateur par ID
POST	/api/users	Crée un nouvel utilisateur
PUT	/api/users/{id}	Met à jour un utilisateur
DELETE	/api/users/{id}	Supprime un utilisateur
GET	/api/users/{id}/orders	Commandes d'un utilisateur

Conseil

Règles de nommage :

- Utiliser des **noms** (pas de verbes) : /users et non /getUsers
- Utiliser le **pluriel** : /users et non /user
- Utiliser des **minuscules** et des tirets : /user-profiles
- Versionner l'API : /api/v1/users

3.2 Format JSON

JSON (JavaScript Object Notation) est le format standard pour les API REST.

```
1 {  
2   "id": 1,  
3   "email": "modou@uad.sn",  
4   "firstName": "Modou",  
5   "lastName": "DIAGNE",  
6   "roles": ["USER", "ADMIN"],  
7   "createdAt": "2024-01-15T10:30:00Z",  
8   "address": {  
9     "city": "Bambey",  
10    "country": "Senegal"  
11  }  
12 }
```

Listing 3.1 – Exemple de réponse JSON

Chapitre 4

Controllers et Services Spring

4.1 Annotations des Controllers

4.1.1 @RestController

```
1 // @RestController = @Controller + @ResponseBody
2 @RestController
3 @RequestMapping("/api/users")
4 public class UserController {
5
6     @GetMapping // Equivalent a @GetMapping("/")
7     public List<User> getAllUsers() {
8         return userService.findAll();
9     }
10 }
11
12 // Equivalent avec @Controller
13 @Controller
14 @RequestMapping("/api/users")
15 public class UserController {
16
17     @GetMapping
18     @ResponseBody // Necessary pour retourner du JSON
19     public List<User> getAllUsers() {
20         return userService.findAll();
21     }
22 }
```

Listing 4.1 – @RestController vs @Controller

4.1.2 Mapping des requêtes

```
1 @RestController
2 @RequestMapping("/api/products")
3 public class ProductController {
4
5     // GET /api/products
```

```

6      @GetMapping
7      public List<Product> getAll() { ... }
8
9      // GET /api/products/123
10     @GetMapping("/{id}")
11     public Product getById(@PathVariable Long id) { ... }
12
13     // GET /api/products/search?name=phone&minPrice=100
14     @GetMapping("/search")
15     public List<Product> search(
16         @RequestParam String name,
17         @RequestParam(required = false, defaultValue = "0")
18             Double minPrice) {
19         ...
20     }
21
22     // POST /api/products
23     @PostMapping
24     public Product create(@RequestBody ProductDTO dto) { ... }
25
26     // PUT /api/products/123
27     @PutMapping("/{id}")
28     public Product update(
29         @PathVariable Long id,
30         @RequestBody ProductDTO dto) { ... }
31
32     // DELETE /api/products/123
33     @DeleteMapping("/{id}")
34     public void delete(@PathVariable Long id) { ... }

```

Listing 4.2 – Annotations de mapping HTTP

4.1.3 ResponseEntity

`ResponseEntity` permet de contrôler la réponse HTTP complète (status, headers, body).

```

1  @RestController
2  @RequestMapping("/api/users")
3  public class UserController {
4
5      private final UserService userService;
6
7      // Injection par constructeur
8      public UserController(UserService userService) {
9          this.userService = userService;
10     }
11
12     // GET - 200 OK
13     @GetMapping

```

```

14     public ResponseEntity<List<User>> getAll() {
15         return ResponseEntity.ok(userService.findAll());
16     }
17
18     // GET by ID - 200 ou 404
19     @GetMapping("/{id}")
20     public ResponseEntity<User> getById(@PathVariable Long id) {
21         return userService.findById(id)
22             .map(ResponseEntity::ok)
23             .orElse(ResponseEntity.notFound().build());
24     }
25
26     // POST - 201 Created avec Location header
27     @PostMapping
28     public ResponseEntity<User> create(@Valid @RequestBody
29         CreateUserDTO dto) {
30         User created = userService.create(dto);
31         URI location = URI.create("/api/users/" + created.getId());
32         return ResponseEntity.created(location).body(created);
33     }
34
35     // PUT - 200 OK
36     @PutMapping("/{id}")
37     public ResponseEntity<User> update(
38         @PathVariable Long id,
39         @Valid @RequestBody UpdateUserDTO dto) {
40         return ResponseEntity.ok(userService.update(id, dto));
41     }
42
43     // DELETE - 204 No Content
44     @DeleteMapping("/{id}")
45     public ResponseEntity<Void> delete(@PathVariable Long id) {
46         userService.delete(id);
47         return ResponseEntity.noContent().build();
48     }

```

Listing 4.3 – Utilisation de ResponseEntity

4.2 La couche Service

4.2.1 Responsabilités du Service

- Contenir la logique métier
- Orchestrer les appels aux repositories
- Gérer les transactions
- Convertir les entités en DTOs

```
1 package com.example.service;
```

```
2
3 import com.example.dto.CreateUserDTO;
4 import com.example.dto.UpdateUserDTO;
5 import com.example.entity.User;
6 import com.example.exception.ResourceNotFoundException;
7 import com.example.repository.UserRepository;
8 import org.springframework.stereotype.Service;
9 import org.springframework.transaction.annotation.Transactional;
10
11 import java.util.List;
12 import java.util.Optional;
13
14 @Service
15 @Transactional
16 public class UserService {
17
18     private final UserRepository userRepository;
19
20     public UserService(UserRepository userRepository) {
21         this.userRepository = userRepository;
22     }
23
24     @Transactional(readOnly = true)
25     public List<User> findAll() {
26         return userRepository.findAll();
27     }
28
29     @Transactional(readOnly = true)
30     public Optional<User> findById(Long id) {
31         return userRepository.findById(id);
32     }
33
34     public User create(CreateUserDTO dto) {
35         // Verifier si l'email existe deja
36         if (userRepository.existsByEmail(dto.getEmail())) {
37             throw new IllegalArgumentException("Email deja utilise"
38                 );
39         }
40
41         User user = new User();
42         user.setEmail(dto.getEmail());
43         user.setFirstName(dto.getFirstName());
44         user.setLastName(dto.getLastName());
45
46         return userRepository.save(user);
47     }
48
49     public User update(Long id, UpdateUserDTO dto) {
50         User user = userRepository.findById(id)
51             .orElseThrow(() -> new ResourceNotFoundException("User"
52                 , id));
```



```
51         user.setFirstName(dto.getFirstName());
52         user.setLastName(dto.getLastName());
53
54         return userRepository.save(user);
55     }
56
57     public void delete(Long id) {
58         if (!userRepository.existsById(id)) {
59             throw new ResourceNotFoundException("User", id);
60         }
61         userRepository.deleteById(id);
62     }
63 }
64 }
```

Listing 4.4 – Exemple de Service

4.3 DTOs (Data Transfer Objects)

Les DTOs sont des objets utilisés pour transférer des données entre les couches.

```
1 // DTO pour la creation
2 public class CreateUserDTO {
3
4     @NotBlank(message = "L'email est obligatoire")
5     @Email(message = "Format d'email invalide")
6     private String email;
7
8     @NotBlank(message = "Le prenom est obligatoire")
9     @Size(min = 2, max = 50)
10    private String firstName;
11
12    @NotBlank(message = "Le nom est obligatoire")
13    @Size(min = 2, max = 50)
14    private String lastName;
15
16    @NotBlank(message = "Le mot de passe est obligatoire")
17    @Size(min = 8, message = "Le mot de passe doit avoir au moins 8
18        caracteres")
19    private String password;
20
21    // Getters et Setters
22 }
23
24 // DTO pour la reponse
25 public class UserDTO {
26
27     private Long id;
28     private String email;
29     private String firstName;
30     private String lastName;
```

```
30     private LocalDateTime createdAt;  
31  
32     // Constructeur depuis l'entite  
33     public static UserDTO fromEntity(User user) {  
34         UserDTO dto = new UserDTO();  
35         dto.id = user.getId();  
36         dto.email = user.getEmail();  
37         dto.firstName = user.getFirstName();  
38         dto.lastName = user.getLastName();  
39         dto.createdAt = user.getCreatedAt();  
40         return dto;  
41     }  
42  
43     // Getters  
44 }
```

Listing 4.5 – Exemple de DTOs

Attention

Pourquoi utiliser des DTOs ?

- Ne pas exposer les entités JPA directement (sécurité)
- Contrôler les données envoyées/reçues
- Éviter les problèmes de lazy loading
- Permettre différentes vues d'une même entité

4.4 Validation avec Bean Validation

```
1 public class ProductDTO {  
2  
3     @NotBlank(message = "Le nom est obligatoire")  
4     @Size(min = 2, max = 100, message = "Le nom doit avoir entre 2  
    et 100 caracteres")  
5     private String name;  
6  
7     @Size(max = 500, message = "Description trop longue")  
8     private String description;  
9  
10    @NotNull(message = "Le prix est obligatoire")  
11    @DecimalMin(value = "0.01", message = "Le prix doit etre  
    positif")  
12    @DecimalMax(value = "99999.99", message = "Prix trop eleve")  
13    private BigDecimal price;  
14  
15    @Min(value = 0, message = "Le stock ne peut pas etre negatif")  
16    private Integer stock;  
17  
18    @Email(message = "Email invalide")
```

```
19     private String supplierEmail;
20
21     @Pattern(regexp = "[A-Z]{2}[0-9]{6}$", message = "Reference
22         invalide")
23     private String reference;
24 }
25
26 // Dans le controller
27 @PostMapping
28 public ResponseEntity<Product> create(@Valid @RequestBody
29     ProductDTO dto) {
30     // Si validation echoue -> MethodArgumentNotValidException ->
31     // 400 Bad Request
32     return ResponseEntity.ok(productService.create(dto));
33 }
```

Listing 4.6 – Annotations de validation

4.5 Questions d'entretien

Question d'entretien

Q1 : Quelle est la différence entre @Controller et @RestController ?

Réponse :

- @Controller : Retourne des vues (HTML, Thymeleaf)
- @RestController : Retourne directement des données (JSON/XML). C'est l'équivalent de @Controller + @ResponseBody.

Question d'entretien

Q2 : Quelle est la différence entre @PathVariable et @RequestParam ?

Réponse :

- @PathVariable : Extrait une valeur de l'URL
Exemple : GET /users/123 → @PathVariable Long id
- @RequestParam : Extrait un paramètre de requête
Exemple : GET /users?page=1&size=10 → @RequestParam int page

Question d'entretien

Q3 : Pourquoi utiliser des DTOs au lieu des entités ?

Réponse :

1. **Sécurité** : Ne pas exposer la structure interne de la base de données
2. **Flexibilité** : Différentes représentations d'une même entité
3. **Performance** : Éviter le lazy loading et les relations non souhaitées
4. **Validation** : Valider les données en entrée avant traitement

Question d'entretien

Q4 : Qu'est-ce qu'une API REST idempotente ?

Réponse : Une opération idempotente produit le même résultat quel que soit le nombre de fois qu'elle est exécutée :

- GET /users/1 : Toujours le même résultat → **Idempotent**
- DELETE /users/1 : Supprime une fois, les appels suivants ne changent rien → **Idempotent**
- POST /users : Crée un nouvel utilisateur à chaque appel → **NON idempotent**

Troisième partie

Persistence des Données avec JPA

Chapitre 5

JPA et Hibernate - Fondamentaux

5.1 Introduction à JPA

5.1.1 Qu'est-ce que JPA ?

JPA (Java Persistence API) est une spécification Java pour la persistance des données. Elle définit un standard pour mapper des objets Java vers des tables de base de données relationnelles (**ORM** - Object-Relational Mapping).

Hibernate est l'implémentation JPA la plus populaire, et c'est celle utilisée par défaut dans Spring Boot.

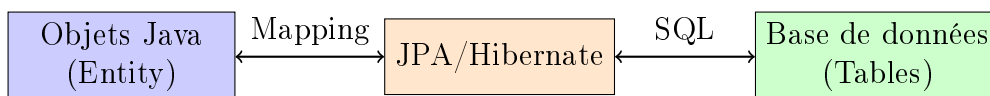


FIGURE 5.1 – Fonctionnement de JPA

5.1.2 Avantages de JPA

1. **Abstraction** : Pas besoin d'écrire de SQL (dans la plupart des cas)
2. **Portabilité** : Changer de base de données facilement
3. **Productivité** : Moins de code boilerplate
4. **Cache** : Gestion automatique du cache de premier niveau
5. **Transactions** : Gestion simplifiée des transactions

5.2 Configuration Spring Data JPA

5.2.1 Dépendances Maven

Listing 5.1 – Dépendances JPA dans pom.xml

```
<!-- Spring Data JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```

</dependency>

<!-- H2 Database (developpement) -->
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>

<!-- PostgreSQL (production) -->
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>

```

5.2.2 Configuration application.properties

Listing 5.2 – Configuration JPA

```

# Configuration H2 (developpement)
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Configuration JPA
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Configuration PostgreSQL (production)
# spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
# spring.datasource.username=postgres
# spring.datasource.password=password
# spring.jpa.hibernate.ddl-auto=validate

```

TABLE 5.1 – Options de hibernate.ddl-auto

Valeur	Description
none	Aucune action
validate	Valide le schéma, ne fait aucune modification
update	Met à jour le schéma (ajoute colonnes/tables)
create	Crée le schéma (supprime les données existantes)
create-drop	Crée au démarrage, supprime à l'arrêt

Attention

En **production**, utilisez toujours `validate` ou `none` et gérez les migrations avec Flyway ou Liquibase.

5.3 Les Entités JPA

5.3.1 Création d'une entité

```
1 package com.example.entity;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 @Entity
7 @Table(name = "users")
8 public class User {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     @Column(name = "email", unique = true, nullable = false, length
15             = 100)
16     private String email;
17
18     @Column(nullable = false)
19     private String password;
20
21     @Column(name = "first_name", length = 50)
22     private String firstName;
23
24     @Column(name = "last_name", length = 50)
25     private String lastName;
26
27     @Column(name = "is_active")
28     private boolean active = true;
29
30     @Column(name = "created_at", updatable = false)
31     private LocalDateTime createdAt;
32
33     @Column(name = "updated_at")
34     private LocalDateTime updatedAt;
35
36     // Callbacks JPA
37     @PrePersist
38     protected void onCreate() {
39         createdAt = LocalDateTime.now();
40         updatedAt = LocalDateTime.now();
41     }
42 }
```

```
40     }
41
42     @PreUpdate
43     protected void onUpdate() {
44         updatedAt = LocalDateTime.now();
45     }
46
47     // Constructeurs
48     public User() {}
49
50     public User(String email, String password) {
51         this.email = email;
52         this.password = password;
53     }
54
55     // Getters et Setters
56     public Long getId() { return id; }
57     public void setId(Long id) { this.id = id; }
58
59     public String getEmail() { return email; }
60     public void setEmail(String email) { this.email = email; }
61
62     public String getPassword() { return password; }
63     public void setPassword(String password) { this.password =
        password; }
64
65     public String getFirstName() { return firstName; }
66     public void setFirstName(String firstName) { this.firstName =
        firstName; }
67
68     public String getLastName() { return lastName; }
69     public void setLastName(String lastName) { this.lastName =
        lastName; }
70
71     public boolean isActive() { return active; }
72     public void setActive(boolean active) { this.active = active; }
73
74     public LocalDateTime getCreatedAt() { return createdAt; }
75     public LocalDateTime getUpdatedAt() { return updatedAt; }
76 }
```

Listing 5.3 – Entité JPA complète

5.3.2 Annotations JPA principales

TABLE 5.2 – Annotations JPA essentielles

Annotation	Description
@Entity	Marque la classe comme entité JPA
@Table	Personnalise le nom de la table
@Id	Définit la clé primaire
@GeneratedValue	Stratégie de génération de l'ID
@Column	Personnalise la colonne (nom, contraintes)
@Transient	Champ non persisté en base
@Temporal	Type temporel (DATE, TIME, TIMESTAMP)
@Enumerated	Persistance des enums (ORDINAL ou STRING)
@Lob	Large Object (texte long, binaire)

5.3.3 Stratégies de génération d'ID

```

1 // AUTO - Hibernate choisit la strategie
2 @GeneratedValue(strategy = GenerationType.AUTO)
3 private Long id;
4
5 // IDENTITY - Auto-increment de la base (MySQL, PostgreSQL)
6 @GeneratedValue(strategy = GenerationType.IDENTITY)
7 private Long id;
8
9 // SEQUENCE - Sequence de la base (Oracle, PostgreSQL)
10 @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "
    user_seq")
11 @SequenceGenerator(name = "user_seq", sequenceName = "user_sequence",
    allocationSize = 1)
12 private Long id;
13
14 // TABLE - Table de sequences (portable mais moins performant)
15 @GeneratedValue(strategy = GenerationType.TABLE)
16 private Long id;
17
18 // UUID - Identifiant unique universel
19 @Id
20 @GeneratedValue(strategy = GenerationType.UUID)
21 private UUID id;

```

Listing 5.4 – Stratégies GenerationType

5.4 Spring Data JPA Repositories

5.4.1 JpaRepository

```
1 package com.example.repository;
2
3 import com.example.entity.User;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.List;
8 import java.util.Optional;
9
10 @Repository
11 public interface UserRepository extends JpaRepository<User, Long> {
12
13     // Methodes heritees de JpaRepository:
14     // - save(entity)
15     // - findById(id)
16     // - findAll()
17     // - deleteById(id)
18     // - count()
19     // - existsById(id)
20
21     // Methodes derivees du nom (Query Methods)
22     Optional<User> findByEmail(String email);
23
24     List<User> findByActiveTrue();
25
26     List<User> findByFirstNameContainingIgnoreCase(String name);
27
28     List<User> findByFirstNameAndLastName(String firstName, String
        lastName);
29
30     boolean existsByEmail(String email);
31
32     long countByActiveTrue();
33
34     void deleteByEmail(String email);
35 }
```

Listing 5.5 – Repository Spring Data JPA

5.4.2 Query Methods - Mots-clés

TABLE 5.3 – Mots-clés pour les Query Methods

Mot-clé	Exemple
And	<code>findByFirstNameAndLastName(String, String)</code>
Or	<code>findByFirstNameOrLastName(String, String)</code>
Between	<code>findByAgeBetween(int, int)</code>
LessThan	<code>findByAgeLessThan(int)</code>
GreaterThan	<code>findByAgeGreaterThan(int)</code>
Like	<code>findByEmailLike(String)</code>
Containing	<code>findByNameContaining(String)</code>
StartingWith	<code>findByNameStartingWith(String)</code>
OrderBy	<code>findByActiveOrderByCreatedAtDesc(boolean)</code>
Not	<code>findByStatusNot(Status)</code>
In	<code>findByStatusIn(Collection<Status>)</code>
True/False	<code>findByActiveTrue()</code>
IsNull	<code>findByDeletedAtIsNull()</code>

5.4.3 Requêtes JPQL personnalisées

```

1 @Repository
2 public interface UserRepository extends JpaRepository<User, Long> {
3
4     // JPQL - Java Persistence Query Language
5     @Query("SELECT u FROM User u WHERE u.email = :email AND u.
6         active = true")
7     Optional<User> findActiveByEmail(@Param("email") String email);
8
9     // JPQL avec JOIN
10    @Query("SELECT u FROM User u JOIN u.orders o WHERE o.total > :
11        minTotal")
12    List<User> findUsersWithOrdersAbove(@Param("minTotal")
13        BigDecimal minTotal);
14
15    // SQL natif
16    @Query(value = "SELECT * FROM users WHERE LOWER(email) LIKE %:
17        pattern%",
18        nativeQuery = true)
19    List<User> searchByEmailNative(@Param("pattern") String pattern
20        );
21
22    // Requete de modification
23    @Modifying
24    @Transactional
25    @Query("UPDATE User u SET u.active = false WHERE u.lastLogin <
26        :date")
27    int deactivateInactiveUsers(@Param("date") LocalDateTime date);

```

```

23 // Requete de suppression
24 @Modifying
25 @Transactional
26 @Query("DELETE FROM User u WHERE u.active = false")
27 void deleteInactiveUsers();
28 }

```

Listing 5.6 – Requêtes JPQL avec @Query

5.5 Pagination et Tri

```

1 @Repository
2 public interface ProductRepository extends JpaRepository<Product,
   Long> {
3
4     // Pagination
5     Page<Product> findByCategory(String category, Pageable pageable
6         );
7
8     // Slice (plus léger que Page)
9     Slice<Product> findByActiveTrue(Pageable pageable);
10 }
11
12 // Utilisation dans le Service
13 @Service
14 public class ProductService {
15
16     public Page<Product> getProducts(int page, int size, String
17         sortBy) {
18         Pageable pageable = PageRequest.of(page, size, Sort.by(
19             sortBy).descending());
20         return productRepository.findAll(pageable);
21     }
22
23     public Page<Product> getProductsByCategory(String category, int
24         page, int size) {
25         Pageable pageable = PageRequest.of(page, size);
26         return productRepository.findByCategory(category, pageable)
27             ;
28     }
29 }
30
31 // Dans le Controller
32 @GetMapping
33 public ResponseEntity<Page<Product>> getProducts(
34     @RequestParam(defaultValue = "0") int page,
35     @RequestParam(defaultValue = "10") int size,
36     @RequestParam(defaultValue = "id") String sortBy) {
37     return ResponseEntity.ok(productService.getProducts(page, size,
38         sortBy));
39 }

```

```
33 }
```

Listing 5.7 – Pagination avec Spring Data

Chapitre 6

Relations JPA

6.1 Types de relations

TABLE 6.1 – Types de relations JPA

Relation	Description	Exemple
@OneToOne	1 :1	User → Profile
@OneToMany	1 :N	User → Orders
@ManyToOne	N :1	Order → User
@ManyToMany	N :N	Student Course

6.2 Relation OneToMany / ManyToOne

```
1 // Cote "One" - User possede plusieurs Orders
2 @Entity
3 @Table(name = "users")
4 public class User {
5
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Long id;
9
10    private String email;
11
12    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL,
13               orphanRemoval = true)
14    private List<Order> orders = new ArrayList<>();
15
16    // Methodes utilitaires pour maintenir la coherence
17    public void addOrder(Order order) {
18        orders.add(order);
19        order.setUser(this);
20    }
21
22    public void removeOrder(Order order) {
```

```

22         orders.remove(order);
23         order.setUser(null);
24     }
25 }
26
27 // Cote "Many" - Order appartient a un User
28 @Entity
29 @Table(name = "orders")
30 public class Order {
31
32     @Id
33     @GeneratedValue(strategy = GenerationType.IDENTITY)
34     private Long id;
35
36     private BigDecimal total;
37
38     @ManyToOne(fetch = FetchType.LAZY)
39     @JoinColumn(name = "user_id", nullable = false)
40     private User user;
41 }

```

Listing 6.1 – Relation OneToMany / ManyToOne

Information

mappedBy indique le côté inverse (non propriétaire) de la relation. La clé étrangère est gérée par l'entité propriétaire (Order dans cet exemple).

6.3 Relation ManyToMany

```

1  @Entity
2  public class Student {
3
4      @Id
5      @GeneratedValue(strategy = GenerationType.IDENTITY)
6      private Long id;
7
8      private String name;
9
10     @ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
11     @JoinTable(
12         name = "student_course",
13         joinColumns = @JoinColumn(name = "student_id"),
14         inverseJoinColumns = @JoinColumn(name = "course_id")
15     )
16     private Set<Course> courses = new HashSet<>();
17
18     // Methodes utilitaires
19     public void enrollCourse(Course course) {

```

```

20     courses.add(course);
21     course.getStudents().add(this);
22 }
23
24 public void dropCourse(Course course) {
25     courses.remove(course);
26     course.getStudents().remove(this);
27 }
28 }
29
30 @Entity
31 public class Course {
32
33     @Id
34     @GeneratedValue(strategy = GenerationType.IDENTITY)
35     private Long id;
36
37     private String title;
38
39     @ManyToMany(mappedBy = "courses")
40     private Set<Student> students = new HashSet<>();
41 }

```

Listing 6.2 – Relation ManyToMany

6.4 Fetch Types

TABLE 6.2 – Stratégies de chargement

Type	Description	Par défaut
EAGER	Charge immédiatement avec l'entité parente	@ManyToOne, @OneToOne
LAZY	Charge uniquement quand accédé	@OneToMany, @ManyToMany

Attention

Problème N+1 : Avec LAZY loading, si vous accédez aux relations dans une boucle, Hibernate exécute N requêtes supplémentaires. Solution : utiliser JOIN FETCH dans les requêtes JPQL.

```

1 @Repository
2 public interface UserRepository extends JpaRepository<User, Long> {
3
4     // Sans JOIN FETCH - Probleme N+1
5     // SELECT * FROM users
6     // puis N x SELECT * FROM orders WHERE user_id = ?
7     List<User> findAll();
8 }

```

```

9      // Avec JOIN FETCH - Une seule requete
10     @Query("SELECT u FROM User u LEFT JOIN FETCH u.orders")
11     List<User> findAllWithOrders();
12
13     // Avec EntityGraph
14     @EntityGraph(attributePaths = {"orders", "orders.items"})
15     @Query("SELECT u FROM User u WHERE u.id = :id")
16     Optional<User> findByIdWithOrdersAndItems(@Param("id") Long id)
17     ;
18 }

```

Listing 6.3 – Résoudre le problème N+1 avec JOIN FETCH

6.5 Cascade Types

TABLE 6.3 – Types de cascade

Type	Description
PERSIST	Sauvegarde aussi les entités enfants
MERGE	Met à jour aussi les entités enfants
REMOVE	Supprime aussi les entités enfants
REFRESH	Rafraîchit aussi les entités enfants
DETACH	Détache aussi les entités enfants
ALL	Toutes les opérations ci-dessus

6.6 Questions d'entretien

Question d'entretien

Q1 : Quelle est la différence entre JPA et Hibernate ?

Réponse :

- **JPA** est une **spécification** (interface/standard)
- **Hibernate** est une **implémentation** de JPA

JPA définit les annotations et interfaces ; Hibernate fournit l'implémentation concrète. Spring Boot utilise Hibernate par défaut.

Question d'entretien

Q2 : Qu'est-ce que le problème N+1 et comment le résoudre ?

Réponse : Le problème N+1 survient quand Hibernate exécute 1 requête principale + N requêtes pour charger les relations.

Solutions :

1. JOIN FETCH dans les requêtes JPQL
2. @EntityGraph pour définir les relations à charger
3. @BatchSize pour charger par lots

Question d'entretien**Q3 : Quelle est la différence entre EAGER et LAZY loading ?****Réponse :**

- **EAGER** : La relation est chargée immédiatement avec l'entité parente (une seule requête avec JOIN)
- **LAZY** : La relation est chargée uniquement quand on y accède (requête supplémentaire)

Recommandation : Utiliser LAZY par défaut et charger explicitement avec JOIN FETCH quand nécessaire.

Question d'entretien**Q4 : À quoi sert l'annotation @Transactional ?****Réponse :** @Transactional gère automatiquement les transactions :

- Ouvre une transaction au début de la méthode
- Commit si succès, Rollback si exception
- Paramètres : `readOnly`, `propagation`, `isolation`, `rollbackFor`

Quatrième partie

Sécurité avec Spring Security

Chapitre 7

Spring Security - Fondamentaux

7.1 Introduction à Spring Security

Spring Security est le framework de sécurité de référence pour les applications Spring. Il fournit :

- **Authentication** : Vérifier l'identité de l'utilisateur
- **Autorisation** : Vérifier les droits d'accès
- **Protection** : CSRF, CORS, XSS, injection SQL

7.1.1 Dépendance Maven

Listing 7.1 – Dépendance Spring Security

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Information

Dès que cette dépendance est ajoutée, **toutes les URLs sont protégées** par défaut. Un mot de passe est généré automatiquement dans la console au démarrage.

7.2 Architecture de Spring Security

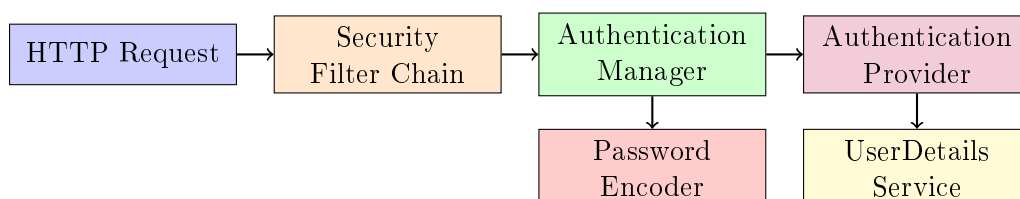


FIGURE 7.1 – Architecture Spring Security

7.2.1 Composants principaux

TABLE 7.1 – Composants de Spring Security

Composant	Rôle
SecurityFilterChain	Chaîne de filtres qui intercepte les requêtes
AuthenticationManager	Gère le processus d'authentification
AuthenticationProvider	Vérifie les credentials (mot de passe)
UserDetailsService	Charge les informations utilisateur depuis la source de données
PasswordEncoder	Encode et vérifie les mots de passe (BCrypt)
SecurityContext	Stocke les informations de l'utilisateur authentifié

7.3 Configuration de base

```

1 package com.example.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
6 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
7 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
8 import org.springframework.security.config.http.SessionCreationPolicy;
9 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
10 import org.springframework.security.crypto.password.PasswordEncoder;
11 import org.springframework.security.web.SecurityFilterChain;
12
13 @Configuration
14 @EnableWebSecurity
15 @EnableMethodSecurity // Active @PreAuthorize, @PostAuthorize
16 public class SecurityConfig {
17
18     @Bean
19     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
20         return http
21             // Desactiver CSRF pour les API REST (stateless)
22             .csrf(csrf -> csrf.disable())
23
24             // Configuration des autorisations
25             .authorizeHttpRequests(auth -> auth
26                 // Endpoints publics
27                 .requestMatchers("/api/auth/**").permitAll()

```

```

28         .requestMatchers("/api/public/**").permitAll()
29         .requestMatchers("/swagger-ui/**", "/v3/api-docs/**")
           .permitAll()
30
31         // Endpoints selon les roles
32         .requestMatchers("/api/admin/**").hasRole("ADMIN")
33         .requestMatchers("/api/users/**").hasAnyRole("USER"
           , "ADMIN")
34
35         // Tout le reste necessite une authentication
36         .anyRequest().authenticated()
37     )
38
39     // Mode stateless pour JWT
40     .sessionManagement(session ->
41         session.sessionCreationPolicy(SessionCreationPolicy
           .STATELESS)
42     )
43
44     .build();
45 }
46
47 @Bean
48 public PasswordEncoder passwordEncoder() {
49     return new BCryptPasswordEncoder();
50 }
51 }

```

Listing 7.2 – Configuration Spring Security

7.4 UserDetailsService

```

1 package com.example.service;
2
3 import com.example.entity.User;
4 import com.example.repository.UserRepository;
5 import org.springframework.security.core.userdetails.UserDetails;
6 import org.springframework.security.core.userdetails.
   UserDetailsService;
7 import org.springframework.security.core.userdetails.
   UsernameNotFoundException;
8 import org.springframework.stereotype.Service;
9
10 @Service
11 public class CustomUserDetailsService implements UserDetailsService
   {
12
13     private final UserRepository userRepository;
14

```

```

15     public CustomUserDetailsService(UserRepository userRepository)
16     {
17         this.userRepository = userRepository;
18     }
19
20     @Override
21     public UserDetails loadUserByUsername(String email)
22         throws UsernameNotFoundException {
23         return userRepository.findByEmail(email)
24             .orElseThrow(() -> new UsernameNotFoundException(
25                 "Utilisateur non trouve: " + email
26             ));
27     }

```

Listing 7.3 – Implémentation de UserDetailsService

7.4.1 Entité User implémentant UserDetails

```

1  @Entity
2  @Table(name = "users")
3  public class User implements UserDetails {
4
5      @Id
6      @GeneratedValue(strategy = GenerationType.IDENTITY)
7      private Long id;
8
9      @Column(unique = true, nullable = false)
10     private String email;
11
12     @Column(nullable = false)
13     private String password;
14
15     @Enumerated(EnumType.STRING)
16     private Role role = Role.USER;
17
18     public enum Role {
19         USER, ADMIN
20     }
21
22     // Implementation de UserDetails
23     @Override
24     public Collection<? extends GrantedAuthority> getAuthorities()
25     {
26         return List.of(new SimpleGrantedAuthority("ROLE_" + role.
27             name()));
28     }
29
30     @Override
31     public String getUsername() {
32         return email; // On utilise l'email comme username

```

```

31     }
32
33     @Override
34     public String getPassword() {
35         return password;
36     }
37
38     @Override
39     public boolean isAccountNonExpired() { return true; }
40
41     @Override
42     public boolean isAccountNonLocked() { return true; }
43
44     @Override
45     public boolean isCredentialsNonExpired() { return true; }
46
47     @Override
48     public boolean isEnabled() { return true; }
49
50     // Getters et Setters standards...
51 }

```

Listing 7.4 – User avec UserDetails

7.5 Autorisation par méthode

```

1  @RestController
2  @RequestMapping("/api/users")
3  public class UserController {
4
5      // Accessible a tous les utilisateurs authentifies
6      @GetMapping("/me")
7      public ResponseEntity<UserDTO> getCurrentUser(
8          @AuthenticationPrincipal User user) {
9          return ResponseEntity.ok(UserDTO.fromEntity(user));
10     }
11
12     // Accessible uniquement aux ADMIN
13     @PreAuthorize("hasRole('ADMIN')")
14     @GetMapping
15     public ResponseEntity<List<UserDTO>> getAllUsers() {
16         return ResponseEntity.ok(userService.findAll());
17     }
18
19     // Accessible au proprietaire ou aux ADMIN
20     @PreAuthorize("hasRole('ADMIN') or #id == authentication.
21         principal.id")
22     @GetMapping("/{id}")
23     public ResponseEntity<UserDTO> getUser(@PathVariable Long id) {
24         return ResponseEntity.ok(userService.findById(id));
25     }
26 }

```

```
24     }
25
26     // Verification personnalisée
27     @PreAuthorize("@securityService.canAccessResource(#id,
28         authentication)")
29     @DeleteMapping("/{id}")
30     public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
31         userService.delete(id);
32         return ResponseEntity.noContent().build();
33     }
34 }
```

Listing 7.5 – Autorisation avec @PreAuthorize

Chapitre 8

Authentication JWT

8.1 Introduction à JWT

JWT (JSON Web Token) est un standard ouvert (RFC 7519) pour transmettre des informations de manière sécurisée entre parties sous forme de token.

8.1.1 Structure d'un JWT

Un JWT est composé de 3 parties séparées par des points :

Listing 8.1 – Structure d'un JWT

```
xxxxx.yyyyy.zzzzz
|       |       |
|       |       +-- Signature
|       +----- Payload (donnees)
+----- Header (algorithme)
```

```
1 // HEADER
2 {
3   "alg": "HS256",
4   "typ": "JWT"
5 }
6
7 // PAYLOAD
8 {
9   "sub": "user@example.com",      // Subject (identifiant)
10  "iat": 1704063600,              // Issued At (date creation)
11  "exp": 1704150000,              // Expiration
12  "roles": ["USER", "ADMIN"]      // Claims personnalisées
13 }
14
15 // SIGNATURE
16 HMACSHA256(
17   base64UrlEncode(header) + "." + base64UrlEncode(payload),
18   secret_key
19 )
```

Listing 8.2 – Contenu d'un JWT décodé

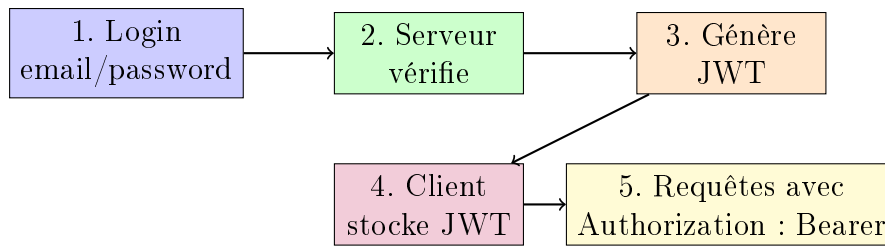


FIGURE 8.1 – Flux d’authentification JWT

8.2 Implémentation JWT

8.2.1 Dépendances

Listing 8.3 – Dépendances JWT (jjwt)

```

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.3</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.3</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.3</version>
  <scope>runtime</scope>
</dependency>

```

8.2.2 Configuration

Listing 8.4 – Configuration JWT

```

# application.properties
jwt.secret=
  MonSecretTresLongDauMoins256BitsPoUrHS256Algorithm123456789
jwt.expiration=86400000
jwt.refresh-expiration=604800000

```

8.2.3 Service JWT

```

1 package com.example.security;
2

```



```
3 import io.jsonwebtoken.*;
4 import io.jsonwebtoken.security.Keys;
5 import org.springframework.beans.factory.annotation.Value;
6 import org.springframework.security.core.userdetails.UserDetails;
7 import org.springframework.stereotype.Service;
8
9 import javax.crypto.SecretKey;
10 import java.util.Date;
11 import java.util.HashMap;
12 import java.util.Map;
13 import java.util.function.Function;
14
15 @Service
16 public class JwtService {
17
18     @Value("${jwt.secret}")
19     private String secret;
20
21     @Value("${jwt.expiration}")
22     private long expiration;
23
24     // Generer un token
25     public String generateToken(UserDetails userDetails) {
26         Map<String, Object> claims = new HashMap<>();
27         claims.put("roles", userDetails.getAuthorities());
28         return createToken(claims, userDetails.getUsername());
29     }
30
31     private String createToken(Map<String, Object> claims, String
        subject) {
32         return Jwts.builder()
33             .claims(claims)
34             .subject(subject)
35             .issuedAt(new Date())
36             .expiration(new Date(System.currentTimeMillis() +
                expiration))
37             .signWith(getSigningKey())
38             .compact();
39     }
40
41     // Extraire le username du token
42     public String extractUsername(String token) {
43         return extractClaim(token, Claims::getSubject);
44     }
45
46     // Extraire la date d'expiration
47     public Date extractExpiration(String token) {
48         return extractClaim(token, Claims::getExpiration);
49     }
50
51     // Extraire un claim specifique
```

```

52     public <T> T extractClaim(String token, Function<Claims, T>
53         claimsResolver) {
54         final Claims claims = extractAllClaims(token);
55         return claimsResolver.apply(claims);
56     }
57
58     private Claims extractAllClaims(String token) {
59         return Jwts.parser()
60             .verifyWith(getSigningKey())
61             .build()
62             .parseSignedClaims(token)
63             .getPayload();
64     }
65
66     // Verifier si le token est valide
67     public boolean isValid(String token, UserDetails
68         userDetails) {
69         final String username = extractUsername(token);
70         return username.equals(userDetails.getUsername()) && !
71             isValidExpired(token);
72     }
73
74     private boolean isValidExpired(String token) {
75         return extractExpiration(token).before(new Date());
76     }
77
78     private SecretKey getSigningKey() {
79         return Keys.hmacShaKeyFor(secret.getBytes());
80     }
81 }

```

Listing 8.5 – JwtService complet

8.2.4 Filtre JWT

```

1 package com.example.security;
2
3 import jakarta.servlet.FilterChain;
4 import jakarta.servlet.ServletException;
5 import jakarta.servlet.http.HttpServletRequest;
6 import jakarta.servlet.http.HttpServletResponse;
7 import org.springframework.security.authentication.
8     UsernamePasswordAuthenticationToken;
9 import org.springframework.security.core.context.
10     SecurityContextHolder;
11 import org.springframework.security.core.userdetails.UserDetails;
12 import org.springframework.security.core.userdetails.
13     UserDetailsServiceImpl;
14 import org.springframework.security.web.authentication.
15     WebAuthenticationDetailsSource;
16 import org.springframework.stereotype.Component;

```

```
13 import org.springframework.web.filter.OncePerRequestFilter;
14
15 import java.io.IOException;
16
17 @Component
18 public class JwtAuthenticationFilter extends OncePerRequestFilter {
19
20     private final JwtService jwtService;
21     private final UserDetailsService userDetailsService;
22
23     public JwtAuthenticationFilter(JwtService jwtService,
24                                   UserDetailsService
25                                   userDetailsService) {
26         this.jwtService = jwtService;
27         this.userDetailsService = userDetailsService;
28     }
29
30     @Override
31     protected void doFilterInternal(HttpServletRequest request,
32                                     HttpServletResponse response,
33                                     FilterChain filterChain)
34         throws ServletException, IOException {
35
36         // Extraire le header Authorization
37         final String authHeader = request.getHeader("Authorization")
38             );
39
40         // Verifier le format "Bearer <token>"
41         if (authHeader == null || !authHeader.startsWith("Bearer "))
42             ) {
43             filterChain.doFilter(request, response);
44             return;
45         }
46
47         // Extraire le token
48         final String jwt = authHeader.substring(7);
49         final String username = jwtService.extractUsername(jwt);
50
51         // Verifier si l'utilisateur n'est pas deja authentifie
52         if (username != null &&
53             SecurityContextHolder.getContext().getAuthentication()
54             == null) {
55
56             UserDetails userDetails = userDetailsService.
57                 loadUserByUsername(username);
58
59             // Valider le token
60             if (jwtService.isTokenValid(jwt, userDetails)) {
61                 UsernamePasswordAuthenticationToken authToken =
62                     new UsernamePasswordAuthenticationToken(
63                         userDetails,
```

```

59         null,
60         userDetails.getAuthorities()
61     );
62
63     authToken.setDetails(
64         new WebAuthenticationDetailsSource().
65             buildDetails(request)
66     );
67
68     // Mettre a jour le SecurityContext
69     SecurityContextHolder.getContext().
70         setAuthentication(authToken);
71
72     }
73
74     filterChain.doFilter(request, response);
75 }
76
77 }

```

Listing 8.6 – JwtAuthenticationFilter

8.2.5 Configuration Security avec JWT

```

1  @Configuration
2  @EnableWebSecurity
3  @EnableMethodSecurity
4  public class SecurityConfig {
5
6      private final JwtAuthenticationFilter jwtAuthFilter;
7      private final AuthenticationProvider authenticationProvider;
8
9      public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter,
10                          AuthenticationProvider
11                          authenticationProvider) {
12
13          this.jwtAuthFilter = jwtAuthFilter;
14          this.authenticationProvider = authenticationProvider;
15      }
16
17      @Bean
18      public SecurityFilterChain securityFilterChain(HttpSecurity
19      http) throws Exception {
20
21          return http
22              .csrf(csrf -> csrf.disable())
23              .authorizeHttpRequests(auth -> auth
24                  .requestMatchers("/api/auth/**").permitAll()
25                  .requestMatchers("/swagger-ui/**", "/v3/api-docs/**")
26                      .permitAll()
27                  .requestMatchers("/api/admin/**").hasRole("ADMIN")
28                  .anyRequest().authenticated()
29              )
30              .sessionManagement(session ->

```

```

26         session.sessionCreationPolicy(SessionCreationPolicy
27             .STATELESS)
28     )
29     .authenticationProvider(authenticationProvider)
30     .addFilterBefore(jwtAuthFilter,
31         UsernamePasswordAuthenticationFilter.class)
32     .build();
33 }
34 }

```

Listing 8.7 – SecurityConfig avec JWT

8.3 Controller d'authentification

```

1  @RestController
2  @RequestMapping("/api/auth")
3  @Tag(name = "Authentication", description = "Endpoints d'
4  authentication")
5  public class AuthController {
6
7      private final AuthService authService;
8
9      public AuthController(AuthService authService) {
10         this.authService = authService;
11     }
12
13     @PostMapping("/register")
14     @Operation(summary = "Inscription d'un nouvel utilisateur")
15     public ResponseEntity<AuthResponse> register(
16         @Valid @RequestBody RegisterRequest request) {
17         return ResponseEntity.ok(authService.register(request));
18     }
19
20     @PostMapping("/login")
21     @Operation(summary = "Connexion utilisateur")
22     public ResponseEntity<AuthResponse> login(
23         @Valid @RequestBody LoginRequest request) {
24         return ResponseEntity.ok(authService.login(request));
25     }
26
27     @PostMapping("/refresh")
28     @Operation(summary = "Rafraichir le token")
29     public ResponseEntity<AuthResponse> refresh(
30         @RequestBody RefreshTokenRequest request) {
31         return ResponseEntity.ok(authService.refreshToken(request));
32     }
33 }
34 // DTOs

```

```
35 public class LoginRequest {
36     @NotBlank @Email
37     private String email;
38
39     @NotBlank
40     private String password;
41 }
42
43 public class AuthResponse {
44     private String accessToken;
45     private String refreshToken;
46     private String tokenType = "Bearer";
47     private long expiresIn;
48 }
```

Listing 8.8 – AuthController

8.4 Questions d'entretien

Question d'entretien

Q1 : Quelle est la différence entre Authentification et Autorisation ?

Réponse :

- **Authentification** : Vérifie *qui* est l'utilisateur (identité)
- **Autorisation** : Vérifie ce que l'utilisateur peut *faire* (permissions)

Spring Security gère les deux : d'abord authentification (JWT, login), puis autorisation (rôles, @PreAuthorize).

Question d'entretien

Q2 : Pourquoi utiliser JWT plutôt que les sessions ?

Réponse :

- **Stateless** : Le serveur n'a pas besoin de stocker l'état de session
- **Scalabilité** : Fonctionne avec plusieurs serveurs sans partage de session
- **Microservices** : Un token peut être validé par plusieurs services
- **Mobile/SPA** : Plus adapté aux clients non-browser

Question d'entretien

Q3 : Comment sécuriser un JWT ?

Réponse :

1. **Secret fort** : Au moins 256 bits pour HS256
2. **Expiration courte** : 15-60 minutes pour l'access token
3. **Refresh token** : Pour renouveler sans re-login
4. **HTTPS** : Toujours transmettre en HTTPS

5. **Stockage client** : HttpOnly cookie plutôt que localStorage
6. **Blacklist** : Pour invalider les tokens (logout)

Question d'entretien

Q4 : Comment fonctionne @PreAuthorize ?

Réponse : @PreAuthorize est une annotation qui vérifie les autorisations *avant* l'exécution de la méthode. Elle utilise SpEL (Spring Expression Language) :

```
1 @PreAuthorize("hasRole('ADMIN')")           // Role ADMIN
2 @PreAuthorize("hasAnyRole('USER', 'ADMIN')") // USER ou
   ADMIN
3 @PreAuthorize("hasAuthority('WRITE_PRIVILEGE')") // Permission
   spécifique
4 @PreAuthorize("#id == authentication.principal.id") //
   Verification custom
```

Nécessite @EnableMethodSecurity dans la configuration.

Cinquième partie

Production et Bonnes Pratiques

Chapitre 9

Documentation API avec OpenAPI

9.1 Introduction à OpenAPI/Swagger

OpenAPI (anciennement Swagger) est un standard pour décrire les API REST. **SpringDoc** est la bibliothèque qui génère automatiquement cette documentation pour Spring Boot.

9.1.1 Dépendance

Listing 9.1 – Dépendance SpringDoc OpenAPI

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.3.0</version>
</dependency>
```

9.1.2 Configuration

Listing 9.2 – Configuration SpringDoc

```
# application.properties
springdoc.api-docs.path=/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
springdoc.swagger-ui.operationsSorter=method
springdoc.swagger-ui.tagsSorter=alpha
```

9.1.3 Configuration Java

```
1 @Configuration
2 public class OpenAPIConfig {
3
4     @Bean
5     public OpenAPI customOpenAPI() {
6         return new OpenAPI()
```

```

7      .info(new Info()
8          .title("API de Gestion de Taches")
9          .version("1.0.0")
10         .description("Documentation de l'API Task Manager")
11         .contact(new Contact()
12             .name("Modou DIAGNE")
13             .email("modou@uad.sn"))
14         .license(new License()
15             .name("Apache 2.0")
16             .url("http://www.apache.org/licenses/LICENSE-2.0")))
17     .addSecurityItem(new SecurityRequirement().addList("bearerAuth"))
18     .components(new Components()
19         .addSecuritySchemes("bearerAuth",
20             new SecurityScheme()
21                 .name("bearerAuth")
22                 .type(SecurityScheme.Type.HTTP)
23                 .scheme("bearer")
24                 .bearerFormat("JWT"))));
25 }
26 }

```

Listing 9.3 – Configuration OpenAPI

9.2 Annotations OpenAPI

```

1  @RestController
2  @RequestMapping("/api/products")
3  @Tag(name = "Products", description = "Gestion des produits")
4  public class ProductController {
5
6      @Operation(
7          summary = "Liste des produits",
8          description = "Retourne tous les produits avec pagination"
9      )
10     @ApiResponses(value = {
11         @ApiResponse(responseCode = "200", description = "Liste des produits",
12             content = @Content(schema = @Schema(implementation = Page.class))),
13         @ApiResponse(responseCode = "401", description = "Non authentifie")
14     })
15     @GetMapping
16     public ResponseEntity<Page<ProductDTO>> getAll(
17         @Parameter(description = "Numero de page")
18         @RequestParam(defaultValue = "0") int page,
19         @Parameter(description = "Taille de page")
20         @RequestParam(defaultValue = "10") int size) {

```

```

21         return ResponseEntity.ok(productService.findAll(page, size)
22             );
23     }
24     @Operation(summary = "Creer un produit")
25     @ApiResponse(responseCode = "201", description = "Produit cree"
26         )
27     @PostMapping
28     public ResponseEntity<ProductDTO> create(
29         @io.swagger.v3.oas.annotations.parameters.RequestBody(
30             description = "Donnees du produit",
31             required = true,
32             content = @Content(schema = @Schema(implementation
33                 = CreateProductDTO.class))
34         )
35         @Valid @RequestBody CreateProductDTO dto) {
36         return ResponseEntity.status(201).body(productService.
37             create(dto));
38     }
39 }

```

Listing 9.4 – Documentation d'un Controller

9.2.1 Documentation des DTOs

```

1 @Schema(description = "Donnees pour creer un produit")
2 public class CreateProductDTO {
3
4     @Schema(
5         description = "Nom du produit",
6         example = "iPhone 15 Pro",
7         minLength = 2,
8         maxLength = 100,
9         required = true
10    )
11    @NotBlank
12    @Size(min = 2, max = 100)
13    private String name;
14
15    @Schema(
16        description = "Prix en euros",
17        example = "1199.99",
18        minimum = "0.01"
19    )
20    @NotNull
21    @DecimalMin("0.01")
22    private BigDecimal price;
23
24    @Schema(
25        description = "Categorie du produit",
26        example = "ELECTRONICS",

```

```
27     allowableValues = {"ELECTRONICS", "CLOTHING", "FOOD"}
28   )
29   private Category category;
30 }
```

Listing 9.5 – Documentation d'un DTO

Chapitre 10

Bonnes Pratiques de Production

10.1 Structure de projet

Listing 10.1 – Structure recommandée

```
src/main/java/com/example/myapp/
|-- MyAppApplication.java
|
|-- config/                                # Configuration
|   |-- SecurityConfig.java
|   |-- OpenAPIConfig.java
|
|-- controller/                            # Couche Presentation
|   |-- AuthController.java
|   |-- UserController.java
|
|-- service/                               # Couche Metier
|   |-- AuthService.java
|   |-- UserService.java
|
|-- repository/                           # Couche Acces Donnees
|   |-- UserRepository.java
|
|-- entity/                               # Entites JPA
|   |-- User.java
|
|-- dto/                                   # Data Transfer Objects
|   |-- request/
|       |-- LoginRequest.java
|   |-- response/
|       |-- UserResponse.java
|
|-- exception/                            # Gestion des erreurs
|   |-- GlobalExceptionHandler.java
|   |-- ResourceNotFoundException.java
|
|-- security/                             # Securite
|   |-- JwtService.java
```

```
| | -- JwtAuthFilter.java
```

10.2 Gestion des erreurs

```
1 @RestControllerAdvice
2 public class GlobalExceptionHandler {
3
4     private static final Logger log = LoggerFactory.getLogger(
5         GlobalExceptionHandler.class);
6
7     @ExceptionHandler(ResourceNotFoundException.class)
8     public ResponseEntity<ErrorResponse> handleNotFound(
9         ResourceNotFoundException ex,
10         HttpServletRequest request) {
11
12         log.warn("Resource not found: {}", ex.getMessage());
13
14         ErrorResponse error = new ErrorResponse(
15             HttpStatus.NOT_FOUND.value(),
16             "Not Found",
17             ex.getMessage(),
18             request.getRequestURI()
19         );
20
21         return ResponseEntity.status(HttpStatus.NOT_FOUND).body(
22             error);
23
24     @ExceptionHandler(MethodArgumentNotValidException.class)
25     public ResponseEntity<ErrorResponse> handleValidation(
26         MethodArgumentNotValidException ex,
27         HttpServletRequest request) {
28
29         Map<String, String> errors = new HashMap<>();
30         ex.getBindingResult().getFieldErrors().forEach(error ->
31             errors.put(error.getField(), error.getDefaultMessage()));
32
33         ErrorResponse errorResponse = new ErrorResponse(
34             HttpStatus.BAD_REQUEST.value(),
35             "Validation Error",
36             "Donnees invalides",
37             request.getRequestURI()
38         );
39         errorResponse.setValidationErrors(errors);
40
41         return ResponseEntity.badRequest().body(errorResponse);
42     }
43 }
```



```

44     @ExceptionHandler(Exception.class)
45     public ResponseEntity<ErrorResponse> handleAll(
46         Exception ex,
47         HttpServletRequest request) {
48
49         log.error("Unexpected error", ex);
50
51         ErrorResponse error = new ErrorResponse(
52             HttpStatus.INTERNAL_SERVER_ERROR.value(),
53             "Internal Server Error",
54             "Une erreur inattendue s'est produite",
55             request.getRequestURI()
56         );
57
58         return ResponseEntity.status(HttpStatus.
59             INTERNAL_SERVER_ERROR).body(error);
60     }

```

Listing 10.2 – GlobalExceptionHandler

10.3 Profils Spring

Listing 10.3 – application-dev.properties

```

# ===== DEV =====
server.port=8080

# H2 Database
spring.datasource.url=jdbc:h2:mem:devdb
spring.h2.console.enabled=true
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=create-drop

# Swagger active
springdoc.swagger-ui.enabled=true

# Logging verbose
logging.level.com.example=DEBUG
logging.level.org.hibernate.SQL=DEBUG

```

Listing 10.4 – application-prod.properties

```

# ===== PROD =====
server.port=${PORT:8080}

# PostgreSQL
spring.datasource.url=${DATABASE_URL}
spring.datasource.username=${DATABASE_USER}
spring.datasource.password=${DATABASE_PASSWORD}

```

```
spring.jpa.show-sql=false
spring.jpa.hibernate.ddl-auto=validate

# Swagger desactive
springdoc.swagger-ui.enabled=false
springdoc.api-docs.enabled=false

# JWT secret (variable d'environnement)
jwt.secret=${JWT_SECRET}

# Logging production
logging.level.root=WARN
logging.level.com.example=INFO
```

10.4 Logging

```
1 import org.slf4j.Logger;
2 import org.slf4j.LoggerFactory;
3
4 @Service
5 public class UserService {
6
7     // Logger statique
8     private static final Logger log = LoggerFactory.getLogger(
9         UserService.class);
10
11     public User findById(Long id) {
12         log.debug("Recherche utilisateur avec ID: {}", id);
13
14         return userRepository.findById(id)
15             .orElseThrow(() -> {
16                 log.warn("Utilisateur non trouve: {}", id);
17                 return new ResourceNotFoundException("User", id);
18             });
19     }
20
21     public User create(CreateUserDTO dto) {
22         log.info("Creation utilisateur: {}", dto.getEmail());
23
24         try {
25             User user = new User();
26             // ...
27             User saved = userRepository.save(user);
28             log.info("Utilisateur cree avec ID: {}", saved.getId());
29             ;
30             return saved;
31         } catch (Exception e) {
32             log.error("Erreur creation utilisateur: {}", dto.
33                 getEmail(), e);
34         }
35     }
36 }
```

```
31         throw e;  
32     }  
33 }  
34 }
```

Listing 10.5 – Bonnes pratiques de logging

TABLE 10.1 – Niveaux de log

Niveau	Usage
TRACE	Très détaillé (debug avancé)
DEBUG	Informations de développement
INFO	Événements normaux importants
WARN	Situations anormales mais gérées
ERROR	Erreurs nécessitant une attention

Chapitre 11

Spring Boot Actuator

11.1 Configuration

Listing 11.1 – Dépendance Actuator

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Listing 11.2 – Configuration Actuator

```
# Endpoints exposes
management.endpoints.web.exposure.include=health,info,metrics

# Details du health check
management.endpoint.health.show-details=when-authorized

# Info
management.info.env.enabled=true
info.app.name=Task Manager API
info.app.version=1.0.0
```

11.2 Endpoints Actuator

TABLE 11.1 – Endpoints Actuator principaux

Endpoint	Description
/actuator/health	État de l'application
/actuator/info	Informations sur l'application
/actuator/metrics	Liste des métriques
/actuator/env	Variables d'environnement
/actuator/loggers	Configuration des logs

11.3 Health Indicator personnalisé

```
1 @Component
2 public class DatabaseHealthIndicator implements HealthIndicator {
3
4     private final DataSource dataSource;
5
6     public DatabaseHealthIndicator(DataSource dataSource) {
7         this.dataSource = dataSource;
8     }
9
10    @Override
11    public Health health() {
12        try (Connection conn = dataSource.getConnection()) {
13            if (conn.isValid(5)) {
14                return Health.up()
15                    .withDetail("database", conn.getMetaData().
16                        getDatabaseProductName())
17                    .withDetail("status", "Connected")
18                    .build();
19            }
20        } catch (SQLException e) {
21            return Health.down()
22                .withDetail("error", e.getMessage())
23                .build();
24        }
25        return Health.down().build();
26    }
27 }
```

Listing 11.3 – Custom HealthIndicator

11.4 Questions d'entretien

Question d'entretien

Q1 : Comment gérer les différents environnements (dev, prod) ?

Réponse : Avec les **profils Spring** :

- application-dev.properties pour le développement
- application-prod.properties pour la production
- Activation via -spring.profiles.active=prod
- Variables d'environnement pour les secrets

Question d'entretien

Q2 : Pourquoi utiliser des DTOs ?

Réponse :

1. **Sécurité** : Ne pas exposer les entités JPA
2. **Flexibilité** : Différentes vues des mêmes données
3. **Validation** : Annotations sur les DTOs en entrée
4. **Performance** : Éviter le lazy loading accidentel
5. **Découplage** : API indépendante du modèle de données

Question d'entretien

Q3 : Comment gérer les erreurs globalement dans Spring Boot ?

Réponse : Avec `@RestControllerAdvice` :

- Centralise la gestion d'erreurs
- `@ExceptionHandler` pour chaque type d'exception
- Réponse d'erreur standardisée (timestamp, status, message)
- Logging approprié des erreurs

Question d'entretien

Q4 : Qu'est-ce que Spring Boot Actuator ?

Réponse : Actuator fournit des endpoints pour le monitoring en production :

- `/health` : État de l'application et dépendances
- `/metrics` : Métriques (CPU, mémoire, requêtes HTTP)
- `/info` : Informations sur l'application

Important de sécuriser ces endpoints en production.

Annexe A

Cheatsheet Spring Boot

Cette annexe regroupe les commandes, annotations et configurations les plus utilisées.

A.1 Annotations principales

A.1.1 Configuration et Composants

Annotation	Description
@SpringBootApplication	Point d'entrée de l'application
@Configuration	Classe de configuration Spring
@Component	Composant générique géré par Spring
@Service	Service métier
@Repository	Accès aux données
@Controller	Contrôleur MVC
@RestController	Contrôleur REST (= @Controller + @ResponseBody)
@Bean	Définit un bean dans une classe @Configuration
@Autowired	Injection de dépendance

A.1.2 Mapping HTTP

Annotation	Description
@RequestMapping	Mapping général (classe ou méthode)
@GetMapping	HTTP GET
@PostMapping	HTTP POST
@PutMapping	HTTP PUT
@DeleteMapping	HTTP DELETE
@PatchMapping	HTTP PATCH
@PathVariable	Paramètre dans l'URL (/users/{id})
@RequestParam	Paramètre de requête (?page=1)
@RequestBody	Corps de la requête (JSON)
@ResponseStatus	Code HTTP de la réponse

A.1.3 JPA / Hibernate

Annotation	Description
@Entity	Classe entité JPA
@Table	Nom de la table
@Id	Clé primaire
@GeneratedValue	Génération automatique de l'ID
@Column	Configuration de colonne
@OneToMany	Relation 1 :N
@ManyToOne	Relation N :1
@ManyToMany	Relation N :N
@JoinColumn	Colonne de jointure
@Transactional	Gestion des transactions
@Query	Requête JPQL personnalisée

A.1.4 Validation

Annotation	Description
@Valid	Active la validation sur l'objet
@NotNull	Ne peut pas être null
@NotBlank	Non null et non vide (String)
@NotEmpty	Non null et non vide (Collection)
@Size(min, max)	Taille min/max
@Min / @Max	Valeur numérique min/max
@Email	Format email valide
@Pattern	Expression régulière

A.1.5 Spring Security

Annotation	Description
@EnableWebSecurity	Active Spring Security
@EnableMethodSecurity	Active @PreAuthorize
@PreAuthorize	Vérification avant exécution
@PostAuthorize	Vérification après exécution
@Secured	Rôles requis (simple)
@AuthenticationPrincipal	Injecte l'utilisateur courant

A.2 Commandes Maven

Listing A.1 – Commandes Maven courantes

```
# Compiler le projet
mvn compile
```

```

# Lancer les tests
mvn test

# Packager l'application
mvn package

# Packager sans tests
mvn package -DskipTests

# Lancer l'application
mvn spring-boot:run

# Lancer avec un profil
mvn spring-boot:run -Dspring-boot.run.profiles=prod

# Nettoyer et rebuild
mvn clean install

```

A.3 Configuration application.properties

Listing A.2 – Configuration complète

```

# ===== SERVER =====
server.port=8080
server.servlet.context-path=/api

# ===== DATABASE =====
# H2 (dev)
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true

# PostgreSQL (prod)
# spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
# spring.datasource.username=postgres
# spring.datasource.password=password

# ===== JPA =====
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# ===== SECURITY =====
jwt.secret=your-256-bit-secret-key-here
jwt.expiration=86400000

# ===== ACTUATOR =====
management.endpoints.web.exposure.include=health,info,metrics

```

```
management.endpoint.health.show-details=when-authorized

# ===== SWAGGER =====
springdoc.swagger-ui.enabled=true
springdoc.swagger-ui.path=/swagger-ui.html

# ===== LOGGING =====
logging.level.root=INFO
logging.level.com.example=DEBUG
logging.level.org.hibernate.SQL=DEBUG
```

A.4 Template de projet

A.4.1 Entité de base

```
1 @Entity
2 @Table(name = "items")
3 public class Item {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @Column(nullable = false)
10    private String name;
11
12    @Column(name = "created_at", updatable = false)
13    private LocalDateTime createdAt;
14
15    @Column(name = "updated_at")
16    private LocalDateTime updatedAt;
17
18    @PrePersist
19    protected void onCreate() {
20        createdAt = LocalDateTime.now();
21        updatedAt = LocalDateTime.now();
22    }
23
24    @PreUpdate
25    protected void onUpdate() {
26        updatedAt = LocalDateTime.now();
27    }
28
29    // Constructeurs, Getters, Setters
30 }
```

Listing A.3 – Template d'entité

A.4.2 Repository

```
1 @Repository
2 public interface ItemRepository extends JpaRepository<Item, Long> {
3
4     Optional<Item> findByName(String name);
5
6     List<Item> findByNameContainingIgnoreCase(String name);
7
8     @Query("SELECT i FROM Item i WHERE i.createdAt > :date")
9     List<Item> findRecentItems(@Param("date") LocalDateTime date);
10 }
```

Listing A.4 – Template de Repository

A.4.3 Service

```
1 @Service
2 @Transactional
3 public class ItemService {
4
5     private final ItemRepository itemRepository;
6
7     public ItemService(ItemRepository itemRepository) {
8         this.itemRepository = itemRepository;
9     }
10
11     @Transactional(readOnly = true)
12     public List<ItemDTO> findAll() {
13         return itemRepository.findAll().stream()
14             .map(ItemDTO::fromEntity)
15             .collect(Collectors.toList());
16     }
17
18     @Transactional(readOnly = true)
19     public ItemDTO findById(Long id) {
20         return itemRepository.findById(id)
21             .map(ItemDTO::fromEntity)
22             .orElseThrow(() -> new ResourceNotFoundException("Item", id));
23     }
24
25     public ItemDTO create(CreateItemDTO dto) {
26         Item item = new Item();
27         item.setName(dto.getName());
28         return ItemDTO.fromEntity(itemRepository.save(item));
29     }
30
31     public void delete(Long id) {
32         if (!itemRepository.existsById(id)) {
```

```

33         throw new ResourceNotFoundException("Item", id);
34     }
35     itemRepository.deleteById(id);
36 }
37 }

```

Listing A.5 – Template de Service

A.4.4 Controller

```

1  @RestController
2  @RequestMapping("/api/items")
3  @Tag(name = "Items", description = "Gestion des items")
4  public class ItemController {
5
6      private final ItemService itemService;
7
8      public ItemController(ItemService itemService) {
9          this.itemService = itemService;
10     }
11
12     @GetMapping
13     @Operation(summary = "Liste tous les items")
14     public ResponseEntity<List<ItemDTO>> getAll() {
15         return ResponseEntity.ok(itemService.findAll());
16     }
17
18     @GetMapping("/{id}")
19     @Operation(summary = "Recupere un item par ID")
20     public ResponseEntity<ItemDTO> getById(@PathVariable Long id) {
21         return ResponseEntity.ok(itemService.findById(id));
22     }
23
24     @PostMapping
25     @Operation(summary = "Cree un nouvel item")
26     public ResponseEntity<ItemDTO> create(@Valid @RequestBody
27         CreateItemDTO dto) {
28         ItemDTO created = itemService.create(dto);
29         URI location = URI.create("/api/items/" + created.getId());
30         return ResponseEntity.created(location).body(created);
31     }
32
33     @DeleteMapping("/{id}")
34     @Operation(summary = "Supprime un item")
35     public ResponseEntity<Void> delete(@PathVariable Long id) {
36         itemService.delete(id);
37         return ResponseEntity.noContent().build();
38     }
39 }

```

Listing A.6 – Template de Controller

Annexe B

Glossaire

API	Application Programming Interface - Interface de programmation
REST	REpresentational State Transfer - Style d'architecture pour les services web
JSON	JavaScript Object Notation - Format d'échange de données
JPA	Java Persistence API - Spécification pour l'ORM en Java
ORM	Object-Relational Mapping - Mapping objet-relationnel
DTO	Data Transfer Object - Objet de transfert de données
JWT	JSON Web Token - Standard pour les tokens d'authentification
CRUD	Create, Read, Update, Delete - Opérations de base sur les données
IoC	Inversion of Control - Principe d'inversion de contrôle
DI	Dependency Injection - Injection de dépendances
CSRF	Cross-Site Request Forgery - Attaque de falsification de requête
CORS	Cross-Origin Resource Sharing - Partage de ressources entre origines
Bean	Objet géré par le conteneur Spring
Starter	Ensemble de dépendances pré-configurées Spring Boot

Annexe C

Ressources supplémentaires

C.1 Documentation officielle

- Spring Boot : <https://spring.io/projects/spring-boot>
- Spring Security : <https://spring.io/projects/spring-security>
- Spring Data JPA : <https://spring.io/projects/spring-data-jpa>
- Spring Guides : <https://spring.io/guides>

C.2 Tutoriels recommandés

- Baeldung : <https://www.baeldung.com/>
- Spring.io Tutorials : <https://spring.io/guides>
- JetBrains Academy : <https://www.jetbrains.com/academy/>

C.3 Outils

- Spring Initializr : <https://start.spring.io>
- Postman : <https://www.postman.com/>
- IntelliJ IDEA : <https://www.jetbrains.com/idea/>
- VS Code + Spring Boot Extension Pack

Références

1. Documentation officielle Spring Boot : <https://spring.io/projects/spring-boot>
2. Spring Guides : <https://spring.io/guides>
3. Baeldung Spring Tutorials : <https://www.baeldung.com/>
4. Documentation Spring Security : <https://spring.io/projects/spring-security>
5. Documentation Spring Data JPA : <https://spring.io/projects/spring-data-jpa>